

DESPLEGAMENT D'APLICACIONS WEB

Práctica 8

Despliegue de endpoint backend

ALUMNO

Stepan Andreev

Curso 2025 / 2026

Práctica 8 — Despliegue de endpoint backend

Nota sobre el entorno

La práctica se ha realizado sobre la máquina virtual `my_ubuntu_server` (Ubuntu 24.04 LTS), accedida por SSH desde el host (`castor909@192.168.1.178`). Durante la ejecución se detectaron dos particularidades del entorno que se resolvieron de forma documentada:

Situación encontrada	Decisión aplicada
El puerto 3000 ya estaba ocupado por un contenedor Gitea existente (<code>docker-proxy</code> en <code>0.0.0.0:3000</code>).	La API se publica en el puerto 3001 ; el <code>proxy_pass</code> de Nginx apunta a <code>127.0.0.1:3001</code> . Gitea sigue operativo.
El puerto 80 estaba servido por Apache2 , no por Nginx (Nginx estaba <code>disabled / inactive</code>).	Se detuvo y deshabilitó Apache2 y se habilitó/arrancó Nginx , que pasa a ser el reverse proxy en <code>:80</code> tal como exige la práctica.

El resto de la práctica sigue el enunciado de forma literal.

1. Comprobación del entorno

Tras conectar por SSH a la VM se documentan las versiones de las herramientas y el estado del servicio heredado de la Práctica 7:

```
node -v
npm -v
nginx -v
systemctl status nodeapp
```

Resultado: `node v24.15.0` , `npm 11.12.1` , `nginx/1.24.0 (Ubuntu)` . El servicio `nodeapp` (servidor de desarrollo Angular de la Práctica 7) existe y está cargado.

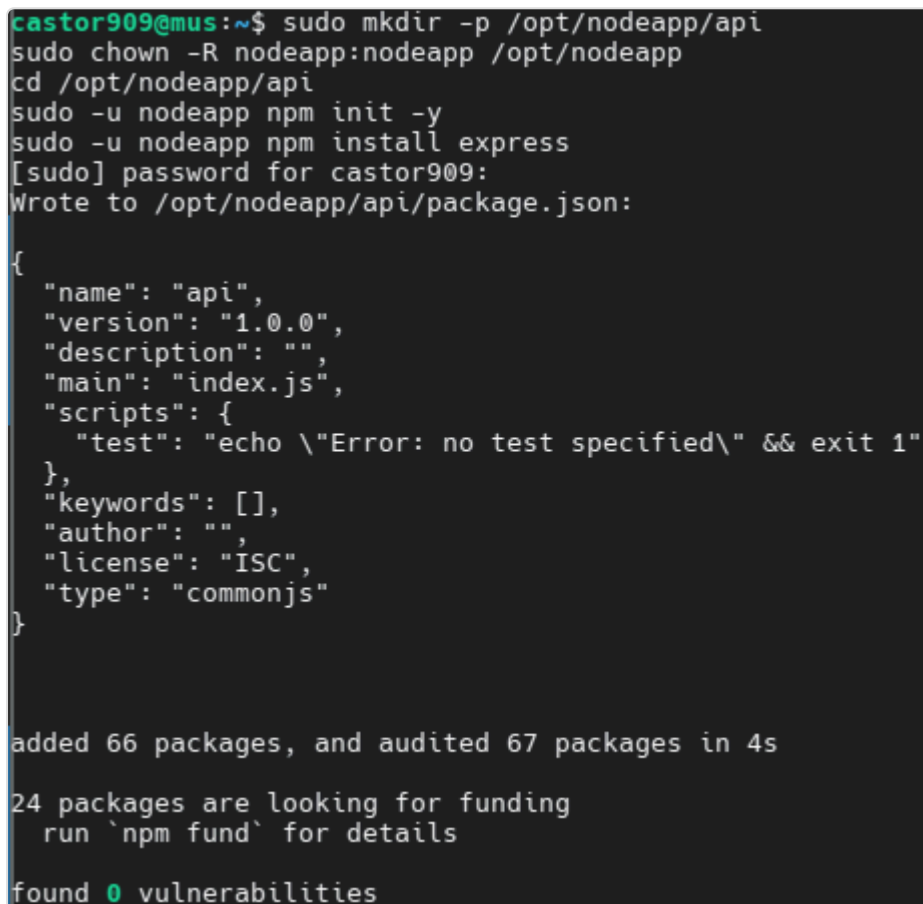
```
castor909@mus:~$ node -v
v24.15.0
castor909@mus:~$ npm -v
11.12.1
castor909@mus:~$ nginx -v
nginx version: nginx/1.24.0 (Ubuntu)
castor909@mus:~$ systemctl status nodeapp
● nodeapp.service - NodeApp (Angular dev server)
   Loaded: loaded (/etc/systemd/system/nodeapp.service; enabled; preset: enabled)
   Active: activating (auto-restart) (Result: exit-code) since Sun 2026-06-14 08:02:38 UTC; 1s ago
     Process: 1862 ExecStart=/usr/bin/npm run start -- --host 0.0.0.0 --port 4200 (code=exited, status=254)
    Main PID: 1862 (code=exited, status=254)
      CPU: 84ms
```

Comprobación del entorno

2. Creación del backend (API)

Se crea el directorio del backend con propietario `nodeapp`, se inicializa el proyecto Node y se instala Express:

```
sudo mkdir -p /opt/nodeapp/api
sudo chown -R nodeapp:nodeapp /opt/nodeapp
cd /opt/nodeapp/api
sudo -u nodeapp npm init -y
sudo -u nodeapp npm install express
```



```
castor909@mus:~$ sudo mkdir -p /opt/nodeapp/api
sudo chown -R nodeapp:nodeapp /opt/nodeapp
cd /opt/nodeapp/api
sudo -u nodeapp npm init -y
sudo -u nodeapp npm install express
[sudo] password for castor909:
Wrote to /opt/nodeapp/api/package.json:

{
  "name": "api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

added 66 packages, and audited 67 packages in 4s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Inicialización del proyecto e instalación de Express

Se crea `server.js` con el endpoint `GET /api/ho!a`. Se usa el puerto **3001** (el 3000 est! ocupado por Gitea). La ruta es `/api/ho!a` porque Nginx har! `proxy_pass` sin reescribir el path, reenviando la URL completa al backend.

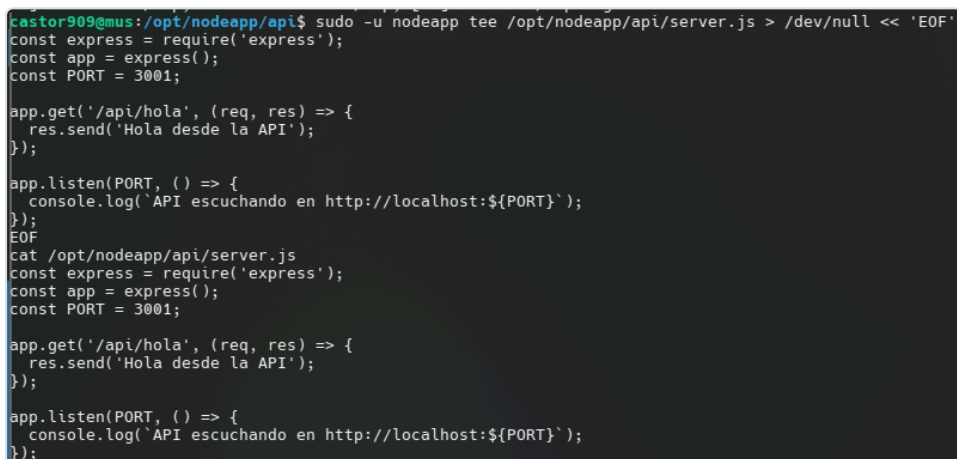
```

sudo -u nodeapp tee /opt/nodeapp/api/server.js > /dev/null << 'EOF'
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});
EOF
cat /opt/nodeapp/api/server.js

```



```

castor909@mus:/opt/nodeapp/api$ sudo -u nodeapp tee /opt/nodeapp/api/server.js > /dev/null << 'EOF'
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});
EOF
cat /opt/nodeapp/api/server.js
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});

```

Contenido de server.js

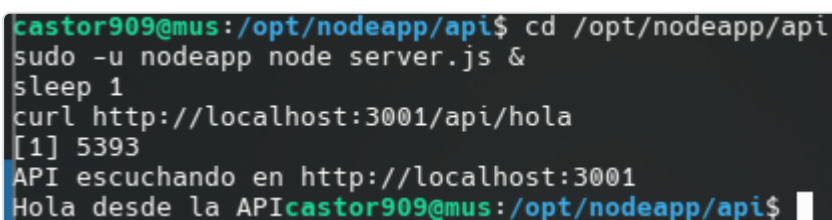
Prueba local del endpoint:

```

cd /opt/nodeapp/api
sudo -u nodeapp node server.js &
sleep 1
curl http://localhost:3001/api/hola

```

La API responde `Hola desde la API`. Tras la prueba se detiene el proceso (`kill %1`) para liberar el puerto.



```

castor909@mus:/opt/nodeapp/api$ cd /opt/nodeapp/api
sudo -u nodeapp node server.js &
sleep 1
curl http://localhost:3001/api/hola
[1] 5393
API escuchando en http://localhost:3001
Hola desde la APIcastor909@mus:/opt/nodeapp/api$

```

Prueba local con curl

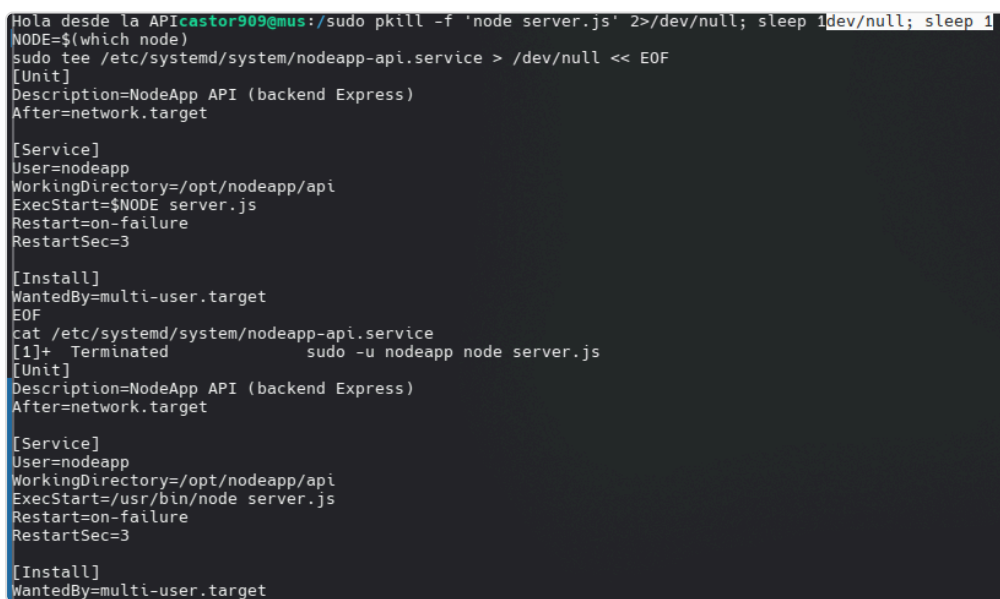
3. Servicio systemd para la API

Se crea la unidad `/etc/systemd/system/nodeapp-api.service`, que ejecuta el backend como usuario `nodeapp`, desde `/opt/nodeapp/api`, y lo reinicia automáticamente si falla (`Restart=on-failure`):

```
NODE=$(which node)
sudo tee /etc/systemd/system/nodeapp-api.service > /dev/null << EOF
[Unit]
Description=NodeApp API (backend Express)
After=network.target

[Service]
User=nodeapp
WorkingDirectory=/opt/nodeapp/api
ExecStart=$NODE server.js
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
cat /etc/systemd/system/nodeapp-api.service
```



```
Hola desde la APIcastor909@mus:/sudo pkill -f 'node server.js' 2>/dev/null; sleep 1dev/null; sleep 1
NODE=$(which node)
sudo tee /etc/systemd/system/nodeapp-api.service > /dev/null << EOF
[Unit]
Description=NodeApp API (backend Express)
After=network.target

[Service]
User=nodeapp
WorkingDirectory=/opt/nodeapp/api
ExecStart=$NODE server.js
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
cat /etc/systemd/system/nodeapp-api.service
[1]+  Terminated                  sudo -u nodeapp node server.js
[Unit]
Description=NodeApp API (backend Express)
After=network.target

[Service]
User=nodeapp
WorkingDirectory=/opt/nodeapp/api
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Unidad systemd

Se activa y arranca el servicio:

```
sudo systemctl daemon-reload
sudo systemctl enable nodeapp-api
sudo systemctl start nodeapp-api
sudo systemctl status nodeapp-api
```

El servicio queda `active (running)` y habilitado en el arranque.

```
castor909@mus:/opt/nodeapp/api$ sudo systemctl daemon-reload
sudo systemctl enable nodeapp-api
sudo systemctl start nodeapp-api
sudo systemctl status nodeapp-api
Created symlink /etc/systemd/system/multi-user.target.wants/nodeapp-api.service → /etc/systemd/system/nodeapp-api.service.
● nodeapp-api.service - NodeApp API (backend Express)
   Loaded: loaded (/etc/systemd/system/nodeapp-api.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-14 08:30:39 UTC; 22ms ago
     Main PID: 6076 (MainThread)
       Tasks: 2 (limit: 2263)
    Memory: 2.4M (peak: 2.5M)
       CPU: 6ms
      CGroup: /system.slice/nodeapp-api.service
              └─6076 /usr/bin/node server.js

Jun 14 08:30:39 mus systemd[1]: Started nodeapp-api.service - NodeApp API (backend Express).
```

Estado del servicio

Consulta de los logs del servicio:

```
sudo journalctl -u nodeapp-api --no-pager -n 20
```

Los logs muestran `Started nodeapp-api.service` y `API escuchando en http://localhost:3001`.

```
castor909@mus:/opt/nodeapp/api$ sudo journalctl -u nodeapp-api --no-pager -n 20
Jun 14 08:30:39 mus systemd[1]: Started nodeapp-api.service - NodeApp API (backend Express).
Jun 14 08:30:39 mus node[6076]: API escuchando en http://localhost:3001
```

Logs del servicio

4. Publicación mediante Nginx

En `/etc/nginx/sites-available/default` se añade, dentro del bloque `server`, un `location /api/` que proxifica hacia el backend (puerto 3001). Se hace copia de seguridad previa del fichero:

```

sudo cp /etc/nginx/sites-available/default /etc/nginx/sites-
available/default.bak
sudo tee /etc/nginx/sites-available/default > /dev/null << 'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;

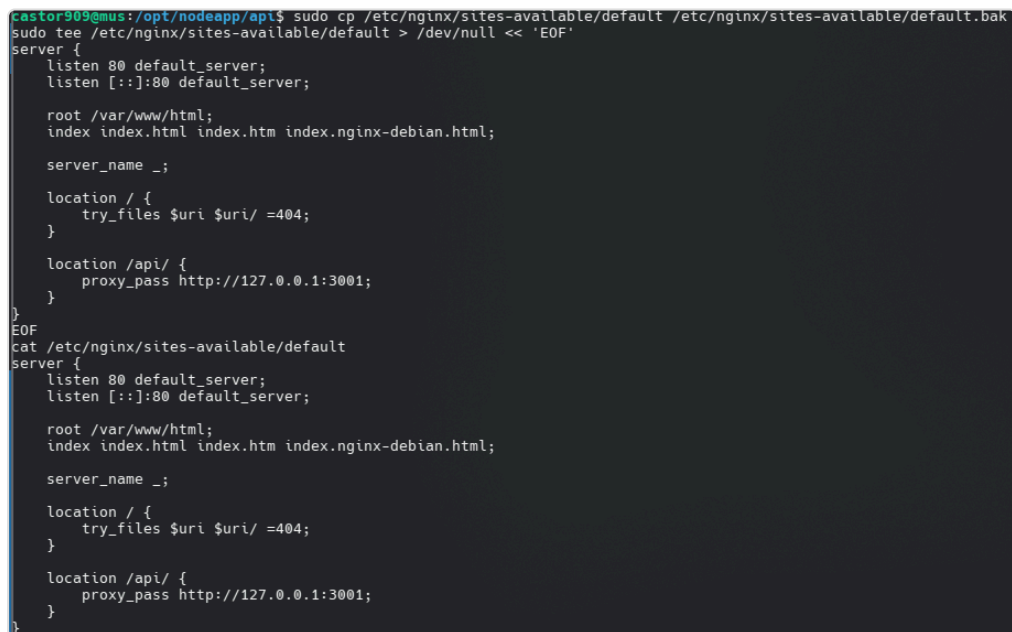
    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:3001;
    }
}
EOF
cat /etc/nginx/sites-available/default

```



```

castor909@mus:/opt/nodeapp/api$ sudo cp /etc/nginx/sites-available/default /etc/nginx/sites-available/default.bak
sudo tee /etc/nginx/sites-available/default > /dev/null << 'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:3001;
    }
}
EOF
cat /etc/nginx/sites-available/default
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:3001;
    }
}

```

Configuración de Nginx con location /api/

Como el puerto 80 estaba servido por Apache2, se conmuta el servidor web a Nginx (deteniendo Apache y habilitando Nginx) y se valida la configuración:

```
sudo systemctl stop apache2
sudo systemctl disable apache2
sudo systemctl enable nginx
sudo systemctl start nginx
sudo nginx -t
sudo systemctl status nginx
```

nginx -t devuelve syntax is ok / test is successful y Nginx queda active (running) sirviendo el puerto 80.

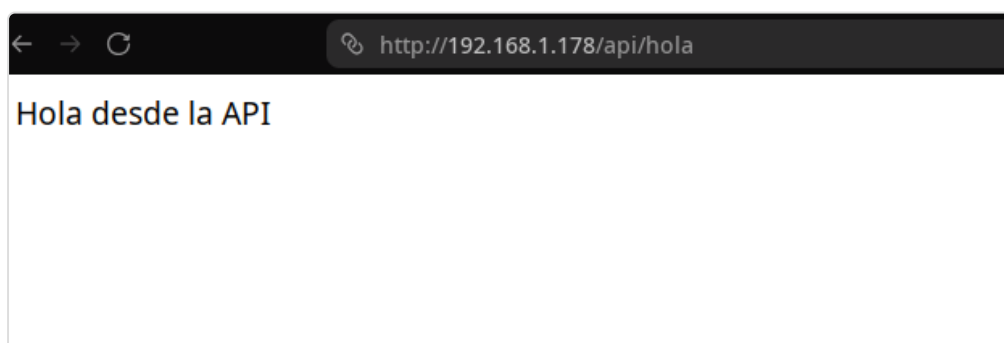
```
laster909@mus:/opt/nodeapp/api$ sudo systemctl stop apache2
sudo systemctl disable apache2
sudo systemctl enable nginx
sudo systemctl start nginx
sudo nginx -t
sudo systemctl status nginx
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable apache2
Removed /etc/systemd/system/multi-user.target.wants/apache2.service.
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-14 08:42:41 UTC; 40ms ago
     Docs: man:nginx(8)
  Process: 8241 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 8243 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 8244 (nginx)
    Tasks: 2 (limit: 2263)
   Memory: 1.7M (peak: 1.9M)
      CPU: 5ms
   CGroup: /system.slice/nginx.service
           └─8244 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─8245 "nginx: worker process"

Jun 14 08:42:41 mus systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jun 14 08:42:41 mus systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

Conmutación y validación de Nginx

Prueba desde el navegador del host a través del reverse proxy:

http://192.168.1.178/api/hola



Acceso a /api/hola desde el navegador

5. Actualización del backend (versión 2)

Se modifica la API: se añade el endpoint /api/hola2 y se actualiza el mensaje de /api/hola para evidenciar la nueva versión:


```

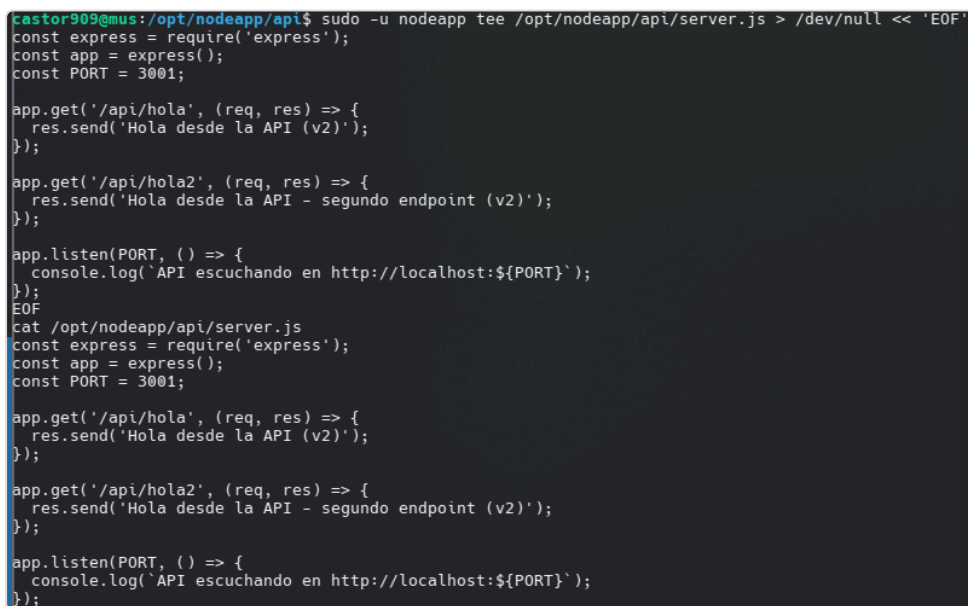
sudo -u nodeapp tee /opt/nodeapp/api/server.js > /dev/null << 'EOF'
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API (v2)');
});

app.get('/api/hola2', (req, res) => {
  res.send('Hola desde la API - segundo endpoint (v2)');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});
EOF
cat /opt/nodeapp/api/server.js

```



```

castor909@mus:/opt/nodeapp/api$ sudo -u nodeapp tee /opt/nodeapp/api/server.js > /dev/null << 'EOF'
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API (v2)');
});

app.get('/api/hola2', (req, res) => {
  res.send('Hola desde la API - segundo endpoint (v2)');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});
EOF
castor909@mus:/opt/nodeapp/api$ cat /opt/nodeapp/api/server.js
const express = require('express');
const app = express();
const PORT = 3001;

app.get('/api/hola', (req, res) => {
  res.send('Hola desde la API (v2)');
});

app.get('/api/hola2', (req, res) => {
  res.send('Hola desde la API - segundo endpoint (v2)');
});

app.listen(PORT, () => {
  console.log(`API escuchando en http://localhost:${PORT}`);
});

```

server.js versión 2

Se empaqueta la aplicación:

```

cd /opt/nodeapp
sudo -u nodeapp tar -czvf nodeapp-api-v2.tar.gz api/
ls -lh nodeapp-api-v2.tar.gz

```

```

api/node_modules/content-type/README.md
api/node_modules/content-type/HISTORY.md
api/node_modules/content-type/LICENSE
api/node_modules/content-type/package.json
api/node_modules/content-type/index.js
api/node_modules/toidentifier/
api/node_modules/toidentifier/README.md
api/node_modules/toidentifier/HISTORY.md
api/node_modules/toidentifier/LICENSE
api/node_modules/toidentifier/package.json
api/node_modules/toidentifier/index.js
api/node_modules/statuses/
api/node_modules/statuses/README.md
api/node_modules/statuses/HISTORY.md
api/node_modules/statuses/LICENSE
api/node_modules/statuses/package.json
api/node_modules/statuses/codes.json
api/node_modules/statuses/index.js
api/node_modules/negotiator/
api/node_modules/negotiator/lib/
api/node_modules/negotiator/lib/mediaType.js
api/node_modules/negotiator/lib/charset.js
api/node_modules/negotiator/lib/encoding.js
api/node_modules/negotiator/lib/language.js
api/node_modules/negotiator/README.md
api/node_modules/negotiator/HISTORY.md
api/node_modules/negotiator/LICENSE
api/node_modules/negotiator/package.json
api/node_modules/negotiator/index.js
api/node_modules/depd/
api/node_modules/depd/lib/
api/node_modules/depd/lib/browser/
api/node_modules/depd/lib/browser/index.js
api/node_modules/depd/LICENSE
api/node_modules/depd/History.md
api/node_modules/depd/Readme.md
api/node_modules/depd/package.json
api/node_modules/depd/index.js
api/node_modules/bytes/
api/node_modules/bytes/LICENSE
api/node_modules/bytes/History.md
api/node_modules/bytes/Readme.md
api/node_modules/bytes/package.json
api/node_modules/bytes/index.js
api/node_modules/safer-buffer/
api/node_modules/safer-buffer/dangerous.js
api/node_modules/safer-buffer/safer.js
api/node_modules/safer-buffer/LICENSE
api/node_modules/safer-buffer/Readme.md
api/node_modules/safer-buffer/package.json
api/node_modules/safer-buffer/Porting-Buffer.md
api/node_modules/safer-buffer/tests.js
api/node_modules/ipaddr.js/
api/node_modules/ipaddr.js/lib/
api/node_modules/ipaddr.js/lib/ipaddr.js.d.ts
api/node_modules/ipaddr.js/lib/ipaddr.js
api/node_modules/ipaddr.js/ipaddr.min.js
api/node_modules/ipaddr.js/README.md
api/node_modules/ipaddr.js/LICENSE
api/node_modules/ipaddr.js/package.json
api/server.js
api/package-lock.json
api/package.json
-rw-rw-r-- 1 nodeapp nodeapp 642K Jun 14 08:52 nodeapp-api-v2.tar.gz
castor909@mus: /opt/nodeapp$

```

Empaquetado de la aplicación

Despliegue de la nueva versión: no hay dependencias nuevas, por lo que no es necesario `npm install` ; basta con reiniciar el servicio:

```
sudo systemctl restart nodeapp-api
sudo systemctl status nodeapp-api --no-pager | head -5
curl http://localhost:3001/api/hola
curl http://localhost:3001/api/hola2
```

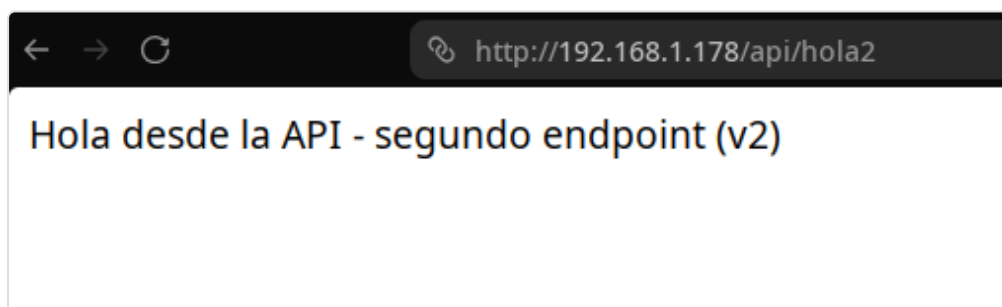
El servicio responde con la nueva versión en ambos endpoints: `Hola desde la API (v2)` y `Hola desde la API - segundo endpoint (v2)`.

```
castor909@mus:/opt/nodeapp$ sudo systemctl status nodeapp-api --no-pager | head -5
curl http://localhost:3001/api/hola
echo
curl http://localhost:3001/api/hola2
● nodeapp-api.service - NodeApp API (backend Express)
   Loaded: loaded (/etc/systemd/system/nodeapp-api.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-14 08:53:47 UTC; 2min 0s ago
     Main PID: 10294 (MainThread)
       Tasks: 7 (limit: 2263)
    Hola desde la API (v2)
    Hola desde la API - segundo endpoint (v2)castor909@mus:/opt/nodeapp$
```

Redespliegue y verificación

Comprobación del nuevo endpoint desde el navegador a través de Nginx:

```
http://192.168.1.178/api/hola2
```



Nuevo endpoint `/api/hola2` desde el navegador

Conclusión

Sobre la infraestructura de la Práctica 7 se ha desplegado un backend Express que expone `/api/hola` y `/api/hola2`, gestionado por un servicio systemd (`nodeapp-api`) con reinicio automático, y publicado al exterior mediante Nginx como reverse proxy en el puerto 80. Se ha demostrado además el ciclo de actualización y redespliegue de una nueva versión de la aplicación.